

Informe de la Pràctica: WebApp amb Redis a AWS

Nerea de la Torre, 1669013 i Adrián Prego, 1672251

Introducció

Aquesta pràctica té com a objectiu implementar una aplicació web que utilitzi Redis com a base de dades en memòria per gestionar productes i cistelles de compra. S'ha desplegat a AWS utilitzant una instància EC2 per a l'aplicació Flask.

Objectiu

L'objectiu principal és configurar una arquitectura que combini una aplicació web amb una base de dades Redis, en un entorn local, assegurant-ne el correcte funcionament i la gestió eficient de dades en temps real.

Fase 1: Inicialització de la Base de Dades

En aquesta fase hem configurat la base de dades Redis amb una llista de productes utilitzant l'script `initialize_redis.py`.

Procés d'implementació

1. **Configuració instància EC2:** Configurem una instància EC2 i llançem la instància amb Amazon Linux 2.
2. **Configuració de Redis:** Hem instal·lat Redis localment a la instància EC2 seguint els passos indicats al document d'instal·lació `HowInstallRedisServer.pdf`:

```
1 sudo yum groupinstall "Development Tools"
  -y
2 sudo yum install gcc jemalloc-devel tcl -y
3 curl -O http://download.redis.io/redis-
  stable.tar.gz
4 tar xzvf redis-stable.tar.gz
5 cd redis-stable
6 make
7 sudo make install
```

Per executar el servidor Redis:

```
1 redis-server --daemonize yes
2 redis-cli ping
```

3. **Instal·lació Python i dependències:** Instal·lem python i utilitzem pip3 per instal·lar les dues biblioteques necessàries: **flask** i **redis**.

```
1 sudo yum install python3 -y
2 pip3 install flask redis
```

4. **Duplicació carpeta WebAppRedis:** Hem creat un repositori a Github i afegit la carpeta WebAppRedis (que inclou els fitxers per crear la web) per tal de facilitar la descàrrega dels fitxers dins la instància. Mitjançant la següent comanda:

```
1 git clone https://github.com/
  nereadelatorre/Practica3.git
```

Una vegada descarregats els fitxers, accedim al directori de la carpeta dins la terminal.

5. **Configurar `initialize_redis.py`** Inicialment, l'arxiu `initialize_redis.py` només contenia la

definició de la funció `initialize_products` amb un comentari tipus TODO, i un bloc principal amb un altre comentari per configurar la connexió a Redis i cridar la funció. A partir d'aquest punt hem realitzat les següents modificacions (el codi complet el podeu trobar a l'Apèndix).

- (a) Vam crear una llista de 10 productes dins de la funció `initialize_products`, cadascun representat com un diccionari amb els camps:
 - **id:** identificador únic del producte
 - **name:** nom del producte
 - **price:** preu decimal
 - **description:** descripció breu
 - **stock:** estoc disponible
- (b) Per cada producte, utilitzem `redis_client.hset()` per guardar-lo a Redis sota la clau `product:{id}`, utilitzant el mètode `mapping=` per afegir totes les propietats de cop.
- (c) Vam afegir la funció `mostrar_productos()`, que permet consultar els productes emmagatzemats mitjançant `redis_client.hgetall()`, que és útil per a validacions i comprovacions manuals.
- (d) Al bloc `if __name__ == "__main__":`, vam configurar la connexió a Redis per a desenvolupament local mitjançant:

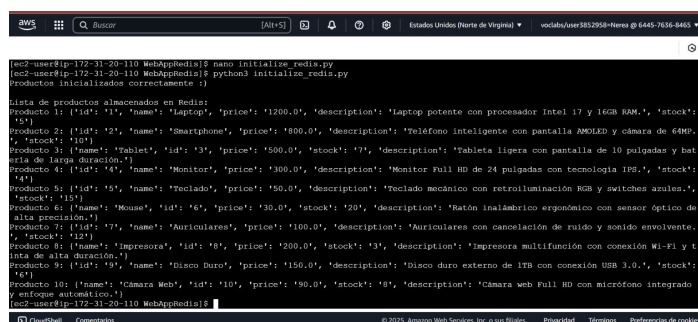
```
1 redis_client = redis.StrictRedis(host=
  "localhost", port=6379,
  decode_responses=True)
```

- (e) A continuació cridem a la funció `initialize_products` per inicialitzar els productes en Redis.
- (f) Finalment, imprimim a la terminal la confirmació de càrrega i el contingut dels productes per verificar correctament l'estat del Redis després de la inicialització.

6. **Execució i validació:** Hem verificat la correcta inserció dels productes executant el fitxer:

```
1 python3 initialize_redis.py
```

Resultats Obtinguts



```
awscli [root@ip-172-31-20-110 ~]# python3 initialize_redis.py
[ec2-user@ip-172-31-20-110 WebAppRedis]# python3 initialize_redis.py
Productos inicializados correctamente.
Lista de productos almacenados en Redis:
Producto 1: {'id': '1', 'name': 'Laptop', 'price': '1200.0', 'description': 'Laptop potente con procesador Intel i7 y 16GB RAM.', 'stock': '50'}
Producto 2: {'id': '2', 'name': 'Smartphone', 'price': '800.0', 'description': 'Teléfono inteligente con pantalla AMOLED y cámara de 64MP.', 'stock': '100'}
Producto 3: {'id': '3', 'name': 'Tablet', 'price': '500.0', 'stock': '75', 'description': 'Tableta ligera con pantalla de 10 pulgadas y batería de larga duración.'}
Producto 4: {'id': '4', 'name': 'Monitor', 'price': '300.0', 'description': 'Monitor Full HD de 24 pulgadas con tecnología IPS.', 'stock': '150'}
Producto 5: {'id': '5', 'name': 'Teclado', 'price': '50.0', 'description': 'Teclado mecánico con retroiluminación RGB y switches azules.', 'stock': '120'}
Producto 6: {'name': 'Mouse', 'id': '6', 'price': '30.0', 'stock': '20', 'description': 'Ratón inalámbrico ergonómico con sensor óptico de alta precisión.'}
Producto 7: {'id': '7', 'name': 'Auriculares', 'price': '100.0', 'description': 'Auriculares con cancelación de ruido y sonido envolvente.', 'stock': '120'}
Producto 8: {'name': 'Impresora', 'id': '8', 'price': '200.0', 'stock': '30', 'description': 'Impresora multifunción con conexión Wi-Fi y lista de alta duración.'}
Producto 9: {'id': '9', 'name': 'Disco Duro', 'price': '150.0', 'description': 'Disco duro externo de 1TB con conexión USB 3.0.', 'stock': '60'}
Producto 10: {'name': 'Cámara Web', 'id': '10', 'price': '90.0', 'stock': '80', 'description': 'Cámara web Full HD con micrófono integrado y enfoque automático.'}
[ec2-user@ip-172-31-20-110 WebAppRedis]#
```

Figure 1. Sortida de l'execució de `initialize_redis.py`

Amb aquesta imatge es pot veure com es carreguen els productes en Redis i el llistat d'aquests.

Fase 2: Implementació de la WebApp

Hem desenvolupat l'aplicació Flask `app.py` que interacciona amb Redis per gestionar productes i cistelles de compra.

Components principals

- **Llistat de productes:** Mostra tots els productes disponibles
- **Cistella de compra:** Permet afegir/eliminar productes
- **Checkout:** Processa la compra i actualitza l'estoc

Funcionament

El fitxer original `app.py` només contenia la declaració de les rutes amb comentaris `TODO`, sense cap funcionalitat implementada. A partir d'aquesta versió inicial, hem dut a terme les següents modificacions (es pot veure el codi sencer a l'apartat de l'Apèndix):

1. Hem configurat correctament la connexió amb Redis mitjançant:

```
1 redis_client = redis.StrictRedis(host='localhost', port=6379, decode_responses=True)
```

2. Hem afegit una variable `USER_ID` amb valor fix `user1` per simular un sistema de sessions d'usuari. Aquesta variable s'utilitza com a part de la clau per identificar la cistella de cada usuari.
3. Implementació completa de la ruta `/products` per recuperar tots els productes de Redis. Fem servir `scan_iter()` per llistar claus que comencen per `product:` i `hgetall()` per obtenir cada producte.
4. A `/cart [GET]`, hem modificat la clau per buscar productes a la cistella utilitzant el prefix `cart:USER_ID:`, permetent així la gestió de cistelles per usuari.
5. La ruta `/cart [POST]` ha estat implementada amb les següents funcionalitats:
 - Comprovació d'existència del producte en Redis.
 - Validació de l'estoc: si l'estoc és 0 o menor, no es permet afegir el producte a la cistella.
 - Si el producte ja és a la cistella, se n'incrementa la quantitat amb `hincrby`.
 - Si no hi és, es crea una nova entrada a la cistella amb id, nom, preu i quantitat inicial de 1.
6. A la ruta `/checkout [POST]`, hem implementat:
 - Recorregut de tots els productes de la cistella.
 - Actualització de l'estoc original del producte a Redis restant la quantitat comprada.
 - Eliminació de totes les entrades de la cistella corresponents a l'usuari.
7. Hem mantingut l'opció `debug=True` per facilitar el desenvolupament i les proves locals.

Abans d'executar l'aplicació Flask, ha estat necessari afegir una regla d'entrada al grup de seguretat de la instància EC2 per permetre el tràfic entrant pel port 5000,

que és el port per defecte on Flask executa l'aplicació. Sense aquesta configuració, no és possible accedir-hi des del navegador.

Un cop feta aquesta configuració, hem executat les comandes següents per inicialitzar les dades i posar en marxa l'aplicació:

```
1 python3 initialize_redis.py
2 python3 app.py
```

Després, hem accedit a la pàgina web mitjançant la IP pública de la instància, afegint el port 5000 a l'URL:

```
1 http://3.80.153.196:5000
```

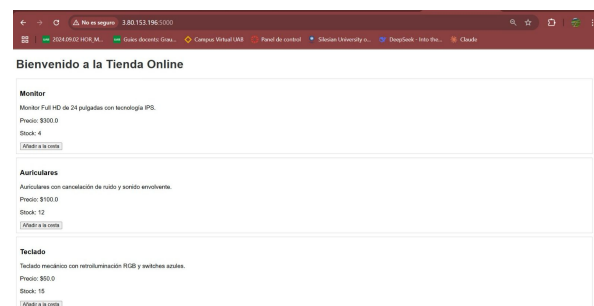


Figure 2. Pàgina principal de l'aplicació Flask un cop iniciada

En aquesta pàgina es mostra el missatge de benvinguda un cop l'aplicació Flask s'ha iniciat correctament. Això indica que la connexió amb el servidor i Redis funciona com s'esperava.

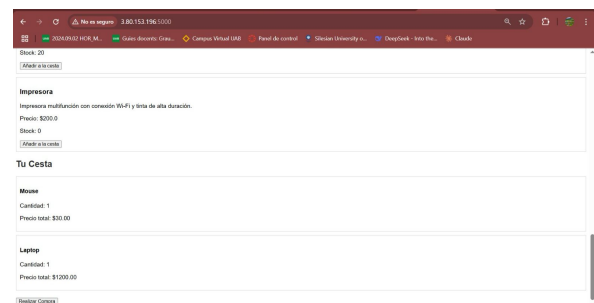


Figure 3. Interfície de visualització de productes

Aquí es llisten tots els productes disponibles, incloent-hi el seu nom, preu i estoc actual. A més, es mostra la secció "Tu cesta", on es desa la informació dels productes que l'usuari ha afegit per comprar. Tota aquesta informació es recupera i actualitza en temps real des de Redis.

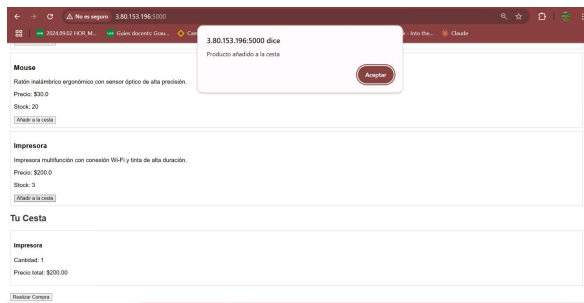


Figure 4. Confirmació d'un producte afegit correctament a la cistella

Després de seleccionar un producte, el sistema confirma que ha estat afegit correctament a la cistella. L'acció incrementa la quantitat si ja existia o el crea si és nou.

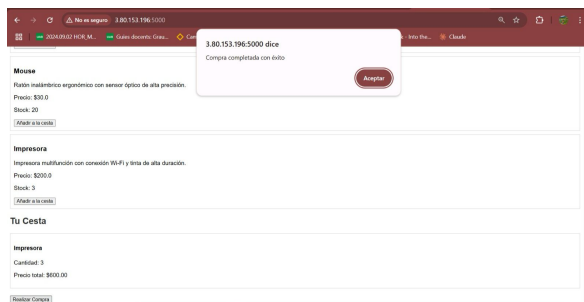


Figure 5. Missatge de compra completada després del checkout

Aquest missatge apareix després de realitzar el checkout. Per poder veure que tots els productes de la cistella es resten del stock i la cistella es buida, cal actualitzar la pàgina al navegador. Ho podem veure en la següent imatge:

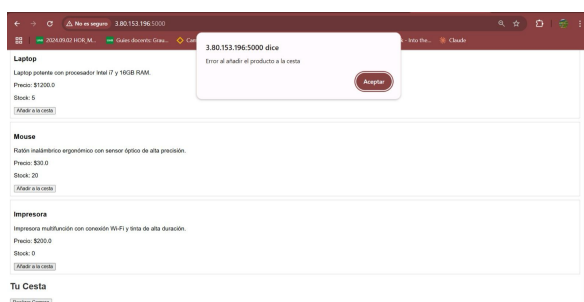


Figure 6. Error al voler afegir un producte sense estoc

En la figura 6 veiem que també s'intenta afegir a la cistella un producte que ja no té estoc (la impresora). L'aplicació detecta la situació i mostra un error informatiu a l'usuari.

Finalitzant així l'experiència completa d'ús de l'aplicació, des de la visualització dels productes fins a la gestió de la cistella i el procés de compra, amb totes les operacions gestionades eficientment mitjançant Redis.

Problemes i Solucions

- **Connexió al servidor web (Flask):** Vam tenir dificultats per accedir a la WebApp des del navegador, ja que les peticions eren bloquejades. Això es va deure a una configuració inadequada del grup de seguretat de la instància EC2. Per resoldre-ho, vam afegir una regla d'entrada que permet connexions externes al port 5000 per protocol TCP.
- **Ús dels fitxers dins la instància EC2:** Inicialment vam trobar complicat traslladar els arxius locals al servidor AWS. Finalment, vam optar per pujar-los a un repositori de GitHub i clonar-lo directament dins de la instància utilitzant la comanda `git clone`, el que ens va simplificar molt el procés.
- **L'estoc no s'actualitzava després del checkout:** En una primera versió, després de completar la compra, l'estoc dels productes no es modificava. El problema es va resoldre recorrent els productes de la cistella i actualitzant l'estoc restant manualment a Redis mitjançant la funció `hincrby()`.
- **Simulació d'usuaris:** En absència d'un sistema d'autenticació, vam simular usuaris mitjançant una variable fixa anomenada `USER_ID`. Tot i que això limita la funcionalitat real, va ser suficient per verificar la gestió de múltiples cistelles de compra de manera independent a Redis.
- **Duplicació d'ítems a la cistella:** Quan s'intentava afegir un producte que ja estava a la cistella, es produïa una duplicació incorrecta. Per evitar-ho, vam implementar una comprovació que, en cas que l'ítem ja existís, incrementa simplement la seva quantitat amb `hincrby()` en lloc de sobrescriure les dades.

Conclusions

Aquesta pràctica ens ha permès comprendre millor el funcionament de Redis com a base de dades en memòria i aprendre a integrar-lo amb una aplicació web Flask per gestionar operacions en temps real. També hem pogut experimentar amb el desplegament en entorns remots mitjançant AWS, configurant instàncies EC2, permisos de xarxa i serveis en segon pla.

Al llarg del desenvolupament, hem enfrontat i superat diversos problemes pràctics, com la configuració del servidor, la inicialització de Redis o la persistència de dades. Tot i les dificultats inicials, hem assolit una aplicació funcional i lleugera, amb una estructura escalable que podria evolucionar fàcilment cap a una versió més avançada amb sistema d'autenticació real o integració amb serveis gestionats com el ElastiCache.

Apèndix

Fitxer initialize_redis.py

```

1 import redis
2
3 def initialize_products(redis_client):
4     productos = [
5         {"id": "1", "name": "Laptop", "price":
6             1200.00, "description": "Laptop potente
7             con procesador Intel i7 y 16GB RAM.", "
8             stock": 5},
9         {"id": "2", "name": "Smartphone", "
10            price": 800.00, "description": "Tel fono
11            inteligente con pantalla AMOLED y c mara
12            de 64MP.", "stock": 10},
13         {"id": "3", "name": "Tablet", "price":
14             500.00, "description": "Tableta ligera
15            con pantalla de 10 pulgadas y bater a de
16            larga duraci n.", "stock": 7},
17         {"id": "4", "name": "Monitor", "price"
18            : 300.00, "description": "Monitor Full HD
19            de 24 pulgadas con tecnolog a IPS.", "
20            stock": 4},
21         {"id": "5", "name": "Teclado", "price"
22            : 50.00, "description": "Teclado mec nico
23            con retroiluminaci n RGB y switches
24            azules.", "stock": 15},
25         {"id": "6", "name": "Mouse", "price":
26            30.00, "description": "Rat n inal mbrico
27            ergon mico con sensor ptico de alta
28            precisi n.", "stock": 20},
29         {"id": "7", "name": "Auriculares", "
30            price": 100.00, "description": "
31            Auriculares con cancelaci n de ruido y
32            sonido envolvente.", "stock": 12},
33         {"id": "8", "name": "Impresora", "
34            price": 200.00, "description": "Impresora
35            multifunci n con conexi n Wi-Fi y tinta
36            de alta duraci n.", "stock": 3},
37         {"id": "9", "name": "Disco Duro", "
38            price": 150.00, "description": "Disco duro
39            externo de 1TB con conexi n USB 3.0.", "
40            stock": 6},
41         {"id": "10", "name": "C mara Web", "
42            price": 90.00, "description": "C mara web
43            Full HD con micr fono integrado y
44            enfoque autom tico.", "stock": 8},
45     ]
46
47     for producto in productos:
48         redis_client.hset(f"product:{producto
49             ['id']}", mapping=producto)
50
51     print("Productos inicializados
52         correctamente :)")
53
54 def mostrar_productos(redis_client):
55     for i in range(1, 11):
56         producto = redis_client.hgetall(f"
57             product:{i}")
58         print(f"Producto {i}: {producto}")
59
60 if __name__ == "__main__":
61     try:
62         redis_client = redis.StrictRedis(host=
63             "localhost", port=6379, decode_responses=
64             True)
65         initialize_products(redis_client)
66         print("\nLista de productos
67             almacenados en Redis:")
68         mostrar_productos(redis_client)
69     except Exception as e:
70         print(f"Error al conectarse a Redis: {
71             e}")

```

Fitxer app.py

```

1 from flask import Flask, jsonify, request,
2     render_template
3 import redis

```

```

4 app = Flask(__name__)
5
6 redis_client = redis.StrictRedis(
7     host='localhost',
8     port=6379,
9     decode_responses=True
10 )
11
12 USER_ID = "user1"
13
14 @app.route('/')
15 def home():
16     return render_template('index.html')
17
18 @app.route('/products', methods=['GET'])
19 def get_products():
20     products = []
21     for key in redis_client.scan_iter("product
22         :*"):
23         product = redis_client.hgetall(key)
24         products.append(product)
25     return jsonify(products)
26
27 @app.route('/cart', methods=['GET'])
28 def get_cart():
29     cart_items = []
30     for key in redis_client.scan_iter(f"cart:{
31         USER_ID}:*"):
32         item = redis_client.hgetall(key)
33         cart_items.append(item)
34     return jsonify(cart_items)
35
36 @app.route('/cart', methods=['POST'])
37 def add_to_cart():
38     data = request.get_json()
39     product_id = data.get('product_id')
40     product_key = f"product:{product_id}"
41
42     if not redis_client.exists(product_key):
43         return jsonify({'status': 'error', '
44             message': 'Producto no encontrado'}), 404
45
46     stock = int(redis_client.hget(product_key,
47         'stock') or 0)
48     if stock <= 0:
49         return jsonify({'status': 'error', '
50             message': 'Producto sin stock'}), 400
51
52     cart_key = f"cart:{USER_ID}:{product_id}"
53     if redis_client.exists(cart_key):
54         redis_client.hincrby(cart_key, '
55             quantity', 1)
56     else:
57         product = redis_client.hgetall(
58             product_key)
59         redis_client.hset(cart_key, mapping={
60             "id": product['id'],
61             "name": product['name'],
62             "price": product['price'],
63             "quantity": 1
64         })
65     return jsonify({'status': 'added'})
66
67 @app.route('/checkout', methods=['POST'])
68 def checkout():
69     for key in redis_client.scan_iter(f"cart:{
70         USER_ID}:*"):
71         cart_item = redis_client.hgetall(key)
72         product_id = cart_item['id']
73         quantity = int(cart_item['quantity'])
74
75         product_key = f"product:{product_id}"
76         if redis_client.exists(product_key):
77             redis_client.hincrby(product_key,
78                 'stock', -quantity)
79
80         redis_client.delete(key)
81
82     return jsonify({'status': 'completed'})
83
84 if __name__ == '__main__':
85     app.run(debug=True, host='0.0.0.0')

```